# Homework 4

Unlike the last two homeworks which all tied into one BIG problem, this one is five mostly separate problems.

> I affirm that I personally wrote the text, code, and comments in this homework assignment.
>
> I received help from [other student] who gave me suggestions on [problem].

- [Aashna Sibal]

## Problem 1

Construct the following `numpy` arrays. For full credit, you should **not** use the code pattern `np.array(my_list)` in any of your answers, nor should you use `for` -loops or any other solution that involves creating or modifying the array one entry at a time.

**Please make sure to show your result so that the grader can evaluate it!**

In [40]:
```
# run this block to import numpy
import numpy as np
```

### (A).

```
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
```

#### Your Solution

In [41]:
```
a = np.arange(0,10).reshape(5,2) #making an array of 0 to 9 and reshaping it to 5 rows, 2 columns
a
```

Out[41]:
```
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
```

## (B).

```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

### Your Solution

In [42]:
```
b = np.arange(0,10).reshape(2,5) #making an array of 0 to 9 and reshaping it to 2 rows, 5 columns
b
```

Out[42]:
```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

## (C).

```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. ])
```

### Your Solution

In [43]:
```
c = np.arange(0,11)/10 #making an array of 0 to 10 and dividing by 10 to get the tenths/floats
c
```

Out[43]:
```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. ])
```

## (D).

```
array([ 1,  1,  1,  1,  1, 10, 10, 10, 10, 10])
```

### Your Solution

In [44]:
```
d = np.arange(0,10) #making an array of 0 to 9 so I can make two masks:
                    #everything less than 5 (or the first 5 numbers) should be replaced by 1
                    #everything greater than 4 (or the last 5 numbers) should be replaced by 10
d[d < 5] = 1
```

```
d[d > 4] = 10
d
```

Out[44]:
```
array([ 1,  1,  1,  1,  1, 10, 10, 10, 10, 10])
```

## (E).

```
array([[30,  1,  2, 30,  4],
       [ 5, 30,  7,  8, 30]])
```

### Your Solution

In [72]:
```
e = np.arange(0,10) #making an array of 0 to 9
e[e%3==0] = 30      #mask so that every number that is divisible by 3 becomes 30
e.reshape(2,5)      #want a 2D array, so reshape to 2 rows, 5 columns
```

Out[72]:
```
array([[30,  1,  2, 30,  4],
       [ 5, 30,  7,  8, 30]])
```

## (F).

```
array([[ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

### Your Solution

In [46]:
```
f = np.arange(5,15).reshape(2,5) #making an array from 5 to 14, reshaping to 2 rows and 5 columns
f
```

Out[46]:
```
array([[ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

## (G).

```
array([[ 1,  3],
       [ 5,  7],
       [ 9, 11],
       [13, 15],
       [17, 19]])
```

### Your Solution

In [47]:
```
g = np.arange(1,20,2).reshape(5,2) #making an array from 1 to 19, but only with every 2nd number,
g                                  #so we want a step of 2; then reshape to 5 rows, 2 columns
```

Out[47]:
```
array([[ 1,  3],
       [ 5,  7],
       [ 9, 11],
       [13, 15],
       [17, 19]])
```

# Problem 2

Consider the following array:

```
A = np.arange(12).reshape(4, 3)
A
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

Construct the specified arrays by indexing `A`. For example, if asked for `array([0, 1, 2])`, a correct answer would be `A[0,:]`. Each of the parts below may be performed in a single line.

In [48]:
```
# run this block to initialize A
A = np.arange(12).reshape(4, 3)
A
```

Out[48]:
```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

## (A).

```
array([6, 7, 8])
```

### Your Solution

In [49]:
```
A[2,:] #indexing only the row labeled 2, or the 3rd row
```

Out[49]:
```
array([6, 7, 8])
```

## (B).

```
array([5, 8])
```

### Your Solution

In [50]:
```
A[1:3,2] #indexing only from row labeled 1, upto and not including 3
         #,2 since we only want the numbers in the column labeled 2
```

Out[50]:
```
array([5, 8])
```

## (C).

```
array([ 6,  7,  8,  9, 10, 11])
```

### Your Solution

In [51]:
```
A[A>5] #making a mask to index only the numbers in the array greater than 5
```

Out[51]:
```
array([ 6,  7,  8,  9, 10, 11])
```

## (D).

```
array([ 0,  2,  4,  6,  8, 10])
```

### Your Solution

In [52]:
```
A[A%2==0] #mask to index only the numbers divisible by 2 in the array (even #s)
```

Out[52]:
```
array([ 0,  2,  4,  6,  8, 10])
```

## (E).

```
array([ 0,  1,  2,  3,  4,  5, 11])
```

### Your Solution

In [53]:
```
A[(A<6)|(A==11)] #mask to get numbers in the array less than 6 or equal to 11
```

Out[53]:
```
array([ 0,  1,  2,  3,  4,  5, 11])
```

## (F).

```
array([ 4, 11])
```

### Your Solution

In [54]:
```
A[(A==4)|(A==11)] #mask to index only 4 and 11 from the array
```

Out[54]:
```
array([ 4, 11])
```

# Problem 3

In this problem, we will use `numpy` array indexing to repair an image that has been artificially separated into multiple pieces. The following code will retrieve two images, one of which has been cutout from the other. Your job is to piece them back together again.

You've already seen `urllib.request.urlopen()` to retrieve online data. We'll play with `mpimg.imread()` a bit more in the future. The short story is that it produces a representation of an image as a `numpy` array of `RGB` values; see below. You'll see `imshow()` a lot more in the near future.

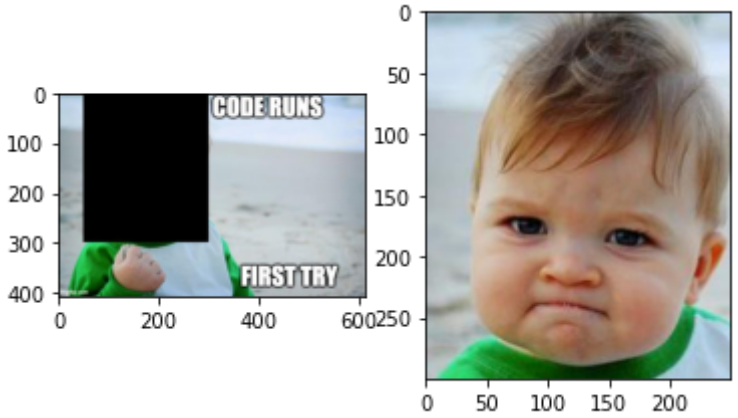
In [55]:
```
import matplotlib.image as mpimg
from matplotlib import pyplot as plt
import urllib

f    = urllib.request.urlopen("https://philchodrow.github.io/PIC16A/homework/main.jpg")
main = mpimg.imread(f, format = "jpg").copy()

f     = urllib.request.urlopen("https://philchodrow.github.io/PIC16A/homework/cutout.jpg")
cutout= mpimg.imread(f, format = "jpg").copy()
```

```
fig, ax = plt.subplots(1, 2)
ax[0].imshow(main)
ax[1].imshow(cutout)
```

Out[55]:
```
<matplotlib.figure.AxesImage at 0x7ff38059e1f0>
```




The images are stored as two `np.array`s `main` and `cutout`. Inspect each one. You'll observe that each is a 3-dimensional `np.array` of shape `(height, width, 3)`. The `3` in this case indicates that the color of each pixel is encoded as an RGB (Red-Blue-Green) value. Each pixel has one RGB value, each of which has three elements.

Use array indexing to fix the image. The result should be that array `main` also contains the data for the face. This can be done just a few lines of carefully-crafted `numpy`. Once you're done, visualize the result by running the indicated code block.

**The black region in `main` starts at row 0, and column 50**. You can learn more about its shape by inspecting the shape of `cutout`.

In [56]:
```
# your solution

A = cutout.shape #A is the shape of the cutout as a tuple

#have to index the part of main that is black and replace it with cutout
#black starts at row 0 and goes till the height of the cutout(300)+0
#for the rows we get: 0:A[0], since height is first in the tuple

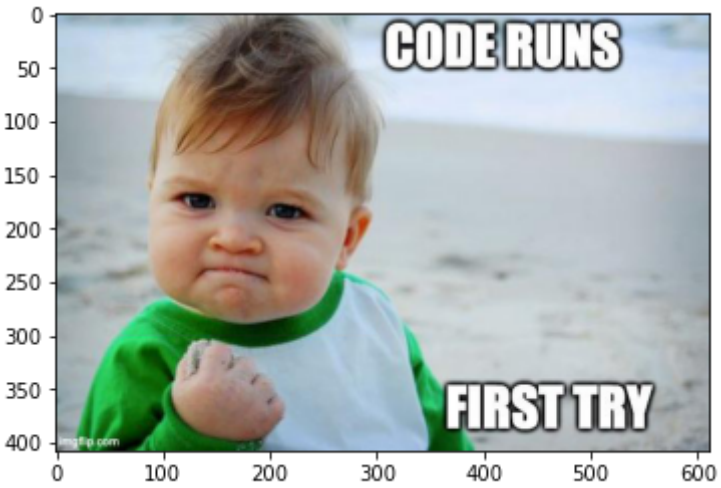
#black starts at column 50 and goes till the width of the cutout(250)+50
#for the rows we get: 50:A[1]+50, since width is second in the tuple

#overall: from the tuple we can see the black in the cutout goes from rows 0-300
#and columns 50-300
```

```
main[0:A[0],50:A[1]+50] = cutout # replacing the appropriate parts of main with cutout
#main[0:300,50:300] = cutout
```

In [57]:
```
# run this block to check your solution
plt.imshow(main)
```

Out[57]:
```
<matplotlib.image.AxesImage at 0x7ff391986220>
```




## Problem 4

You are now about half way through the quarter. Congratulations! I would like you to take this time to reflect on some ways that Python is different than C++. (If you don't know C++, you may use another language instead.)

In [58]:
```
# write difference 1 here:

# In C++, you use {} to create a scope for functions and loops.
# In Python, you can just use a colon and then indent the lines inside the function or loop.
```

In [59]:
```
# write difference 2 here:

# In C++, when you declare any new variable, you need to declare the type too.
# In Python, you do not need to declare the variable type with the variable.
# int x = 5 vs. x = 5
```

In [60]:
```
# write difference 3 here:

# In C++, you have to specify return types and input types in functions.
# In Python, you do not have to. If you do not specify a return, python automatically adds return None.
```

# Problem 5

Recall the simple random walk. At each step, we flip a fair coin. If heads, we move "foward" one unit; if tails, we move "backward."

## (A).

Way back in Homework 1, you wrote some code to simulate a random walk in Python.

Start with this code, or use posted solutions for HW1. If you have since written random walk code that you prefer, you can use this instead. Regardless, take your code, modify it, and enclose it in a function `rw()`. This function should accept a single argument `n`, the length of the walk. The output should be a list giving the position of the random walker, starting with the position after the first step. For example,

```
rw(5)
[1, 2, 3, 2, 3]
```

Unlike in the HW1 problem, you should not use upper or lower bounds. The walk should always run for as long as the user-specified number of steps `n`.

Use your function to print out the positions of a random walk of length `n = 10`.

Don't forget a helpful docstring!

In [61]:
```
import random
```

In [62]:
```
# solution (with demonstration) here

def rw(n):
    """
    A function that produces a random walk with random steps of 1 or -1 each time.
    Returns positions, a list that gives the log of the positions of the walk, starting with the position
    after the first step.
    parameter 1: n, which represents the length of the walk
```

```
    """
    steps = []     # list of history of all step directions
    positions = [] # list of history of all positions
    p = 0          # current position, starts at 0

    while True: # until the break statement is satisfied
        x = random.choice([-1, 1]) # flip a coin to see which step to take
        p += x # add new step to current position
        steps.append(x) # add step direction to log of steps list
        positions.append(p) # record walk position in log list

        if len(positions)==n: # if the length of the list equals n, we want to
            break             # break out of the function since n is the number of steps we want
                              # and each number in positions represents a step

    return positions # return the list of the walk positions log

rw(10) # testing out the function with 10 steps
```

Out[62]:
```
[-1, 0, 1, 0, -1, 0, -1, 0, 1, 2]
```

## (B).

Now create a function called `rw2(n)`, where the argument `n` means the same thing that it did in Part A. Do so using `numpy` tools. Demonstrate your function as above, by creating a random walk of length 10. You can (and should) return your walk as a `numpy` array.

**Requirements**:

- No for-loops.
- This function is simple enough to be implemented as a one-liner of fewer than 80 characters, using lambda notation. Even if you choose not to use lambda notation, the body of your function definition should be no more than three lines long. Importing `numpy` does not count as a line.
- A docstring is required if and only if you take more than one line to define the function.

**Hints**:

- Check the documentation for `np.random.choice()`.
- Discussion 9, and `np.cumsum()`.

In [63]:
```
import numpy as np
```

In [69]:
```
# solution (with demonstration) here
rw2 = lambda n: np.cumsum(np.random.choice([-1,1],n))
# function name is rw2
# lambda function, so we put lambda
# n is the only input
# making a numpy array of random choices of 1 or -1, for size n
# need sum of each step and all the ones before it as they happen to get position
# so we need a cumulative sum for each step in the array

rw2(10) # testing out function for 10 steps
```

Out[69]:
```
array([ 1,  0,  1,  0, -1, -2, -1,  0, -1,  0])
```

## (C).

Use the `%timeit` magic macro to compare the runtime of `rw()` and `rw2()`. Test how each function does in computing a random walk of length `n = 10000`.

In [65]:
```
import timeit
```

In [66]:
```
# solution (with demonstration) here

%timeit rw(10000) #testing time for first function with 10000 steps
%timeit rw2(10000) #testing time for second function with 10000 steps
```

```
6.3 ms ± 20.4 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
90.5 µs ± 260 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

## (D).

Write a few sentences in which you comment on (a) the performance of each function and (b) the ease of writing and reading each function.

(a) Microseconds are much smaller than miliseconds. 6 ms (rw) is around 6000 microseconds, which is much more than 90.8 microseconds (rw2). Therefore, we can see that the second function, rw2, was much faster than rw, since it took less time to run per loop.

(b) The combination of using numpy arrays and a lambda function made the second function much easier to both read and write. For example, rw is around 20 lines long, whereas rw2 is only 1 line long--a big difference. Overall, rw2 is easier to understand when reading and while writing for this reason also.

# (E).

In this problem, we will perform a `d`-dimensional random walk. There are many ways to define such a walk. Here's the definition we'll use for this problem:

> At each timestep, the walker takes one random step forward or backward **in each of `d` directions.**

For example, in a two-dimensional walk on a grid, in each timestep the walker would take a step either north or south, and then another step either east or west. Another way to think about is as the walker taking a single "diagonal" step either northeast, southeast, southwest, or northwest.

Write a function called `rw_d(n,d)` that implements a `d`-dimensional random walk. `n` is again the number of steps that the walker should take, and `d` is the dimension of the walk. The output should be given as a `numpy` array of shape `(n,d)`, where the `k`th row of the array specifies the position of the walker after `k` steps. For example:

```
P = rw_d(5, 3)
P
```

```
array([[-1, -1, -1],
       [ 0, -2, -2],
       [-1, -3, -3],
       [-2, -2, -2],
       [-1, -3, -1]])
```

In this example, the third row `P[2,:] = [-1, -3, -3]` gives the position of the walk after 3 steps.

Demonstrate your function by generating a 3d walk with 5 steps, as shown in the example above.

All the same requirements and hints from Part B apply in this problem as well. It should be possible to solve this problem by making only a few small modifications to your solution from Part B. If you are finding that this is not possible, you may want to either (a) read the documentation for the relevant `numpy` functions more closely or (b) reconsider your Part B approach.

In [70]:
```
# solution (with demonstration) here
rw_d = lambda n,d: (np.cumsum(np.random.choice([-1,1],(n,d)),axis=0))

# function name is rw_d
# lambda function, so we put lambda
# n and d are inputs

# making a numpy array of random choices of 1 or -1, for size (n,d), because we want a 2D array now
# same approach for using cumusm as rw2 (below), except we want to sum over rows since each row
# of the original array is a step in the walk, so we write axis=0

# cumsum explanation:
# need sum of each step and all the ones before it as they happen to get position
# so we need a cumulative sum for each step (row) in the array

P = rw_d(5,3) # testing out the function for a walk of 5 steps with a dimension of 3
print(P)      # printing the resulting array
print(P[2,:]) # printing the position of the walk after 3 steps
```

```
[[ 1 -1 -1]
 [ 0  0 -2]
 [ 1  1 -1]
 [ 0  2 -2]
 [ 1  3 -1]]
[ 1  1 -1]
```

## (F).

In a few sentences, describe how you would have solved Part E without `numpy` tools. Take a guess as to how many lines it would have taken you to define the appropriate function. Based on your findings in Parts C and D, how would you expect its performance to compare to your `numpy`-based function from Part E? Which approach would your recommend?

Note: while I obviously prefer the `numpy` approach, it is reasonable and valid to prefer the "vanilla" way instead. Either way, you should be ready to justify your preference on the basis of writeability, readability, and performance.

Without numpy tools, we would have had to make a more complicated version of rw from part a with an array of arrays, and a while loop to make the arrays after the random choice is picked. We would also need a loop to add the cumulative sums, probably making the code reach at least 20-30 lines. Based on the previous parts of the assignment, defining the function without numpy would probably make the function much slower than the approach I used here in part E. I would prefer/recommend the the numpy approach just because it is shorter, making it easier to read and write, and also faster.

## (G).

Once you've implemented `rw_d()`, you can run the following code to generate a large random walk and visualize it.

```
from matplotlib import pyplot as plt

W = rw_d(20000, 2)
plt.plot(W[:,0], W[:,1])
```

You may be interested in looking at several other visualizations of multidimensional random walks on Wikipedia. Your result in this part will not look exactly the same, but should look qualitatively fairly similar.

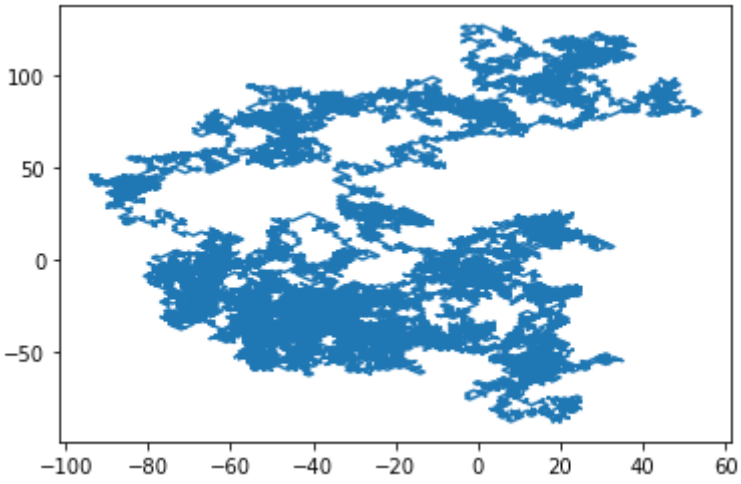
You only need to show one plot. If you like, you might enjoy playing around with the plot settings. While `ax.plot()` is the normal method to use here, `ax.scatter()` with partially transparent points can also produce some intriguing images.

In [71]:
```
# solution (with demonstration) here

from matplotlib import pyplot as plt

W = rw_d(20000, 2)
plt.plot(W[:,0], W[:,1])
```

Out[71]:
```
[<matplotlib.lines.Line2D at 0x7ff36184c5b0>]
```




In [ ]: